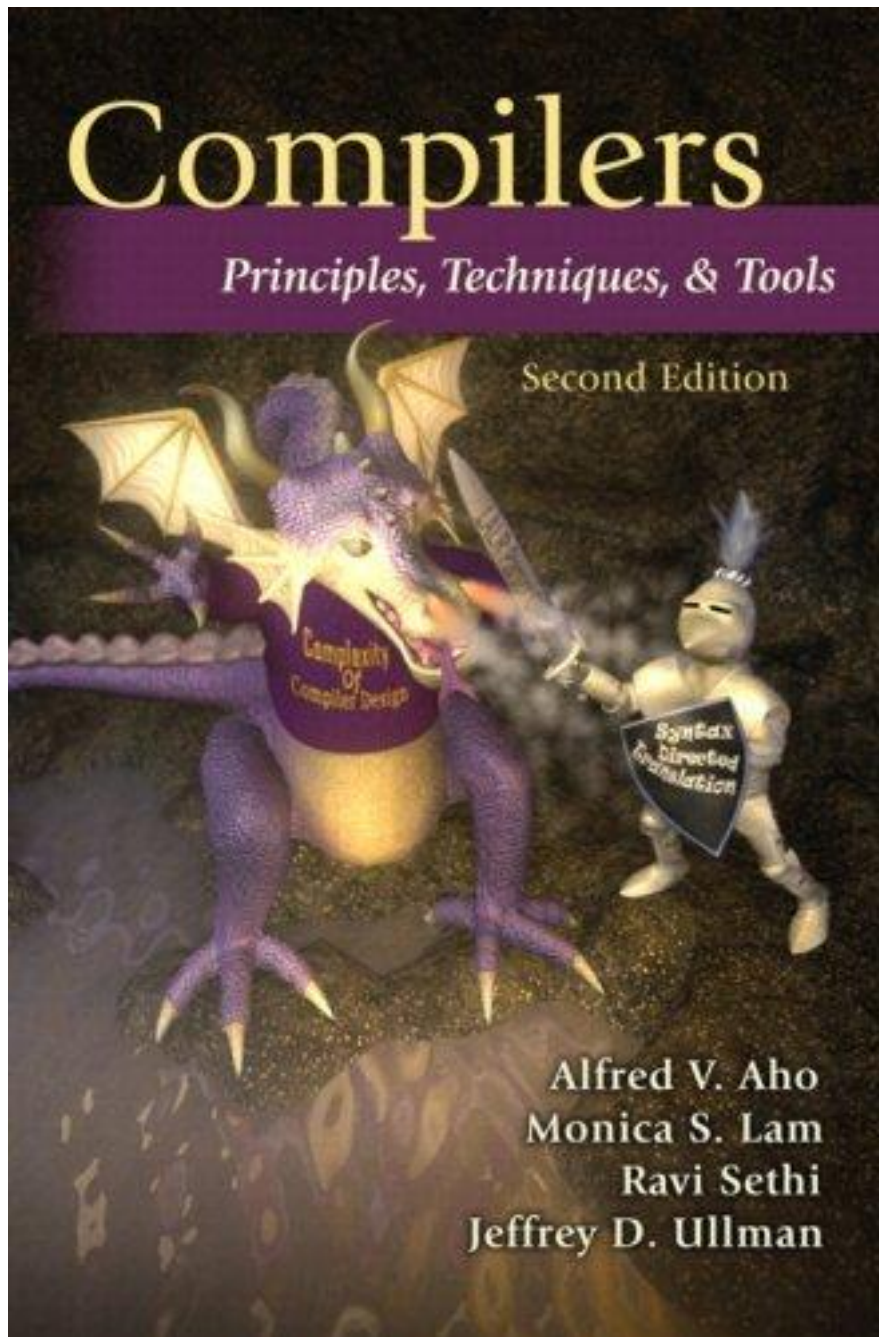




四川大學
SICHUAN UNIVERSITY

Principle of Compiler

luoyining@scu.edu.cn

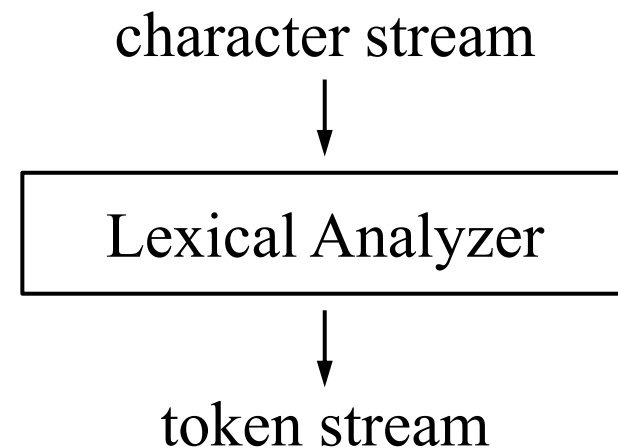


Chapter 3

Lexical Analysis

Chapter 3

- 正则表达式
- 有穷自动机 + 有穷自动机的代码实现
- 正则表达式 → 有穷自动机（三个算法）



CONTENTS

3.1 The Role of the Lexical Analyzer

3.2 Input Buffering

3.3 Specification of Tokens

3.4 Recognition of Tokens

3.6 Finite Automata

3.7 From Regular Expressions to Automata

3.9 Optimization of DFA-Based Pattern Matchers

3.1 The Role of the Lexical Analyzer

Tasks

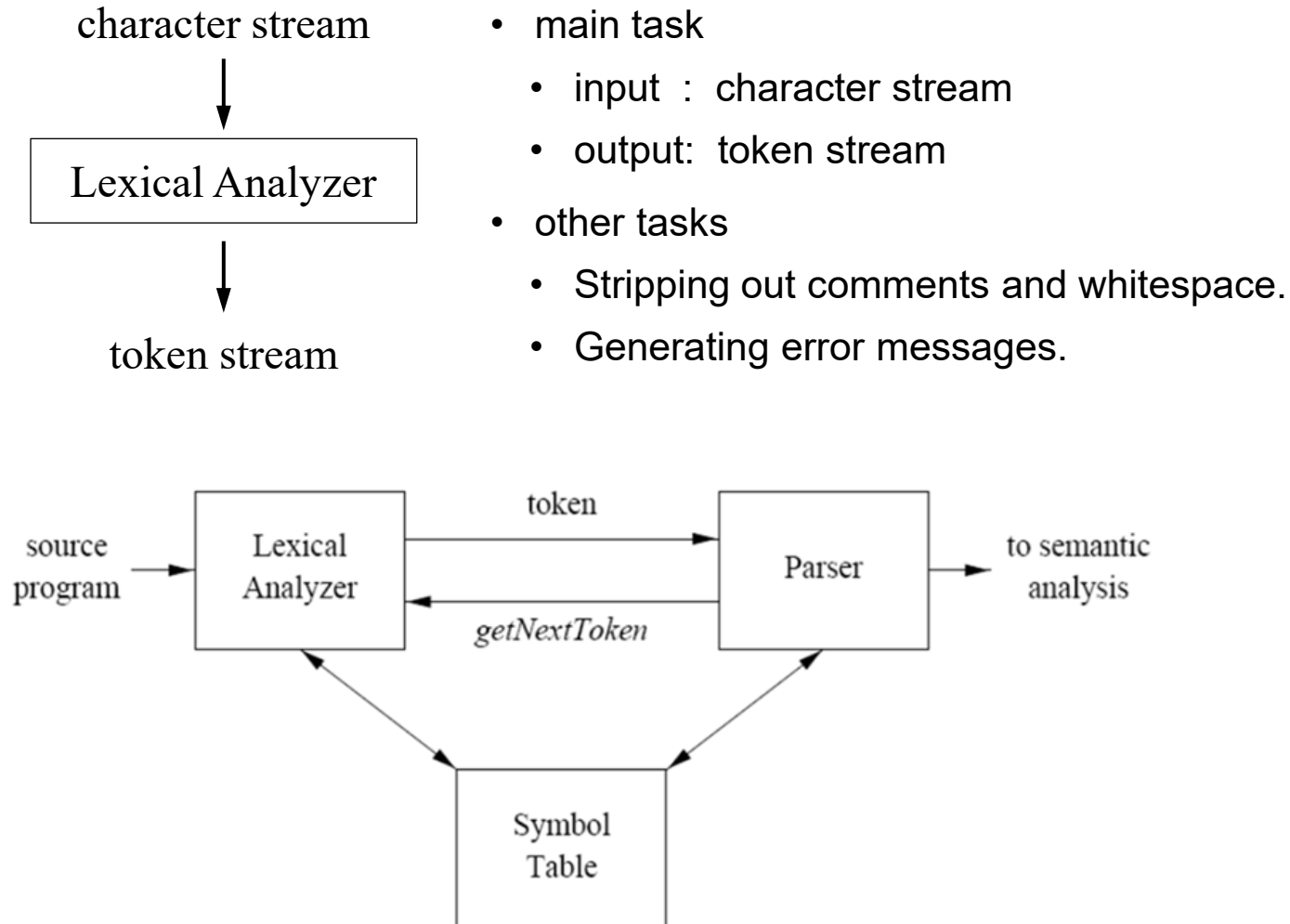


Figure 3.1: Interactions between the lexical analyzer and the parser

```
#define NO_PARSE TRUE
#if NO_PARSE
    while (getToken()!=ENDFILE);
#else
    syntaxTree = parse();
```

```
#define NO_PARSE FALSE
#if NO_PARSE
    while (getToken()!=ENDFILE);
#else
    syntaxTree = parse();
```

```
TreeNode * parse(void)
{
    TreeNode * t;
    token = getToken();
    .....
    return t;
}
```

3.1.1 Lexical Analysis Versus Parsing

“ Why the analysis portion of a compiler is normally separated into lexical analysis and syntax analysis (parsing) phases? ”

1. Simplicity of design is the most important consideration.
2. Compiler efficiency is improved.
3. Compiler portability is enhanced.

3.1.2 Tokens, Patterns, and Lexemes

```
if (n>0) position = initial + rate * 60;
```

Lexeme	Token	Token
position	<id, "position">	<id, 1>
=	<=>	<=>
initial	<id, "initial">	<id, 2>
+	<+>	<+>
rate	<id, "rate">	<id, 3>
*	<*>	<*>
60	<NUM, 60>	<60>

```
<id,1> <=> <id,2> <+> <id,3> <*> <60>
```

- **Token**
 - <token-name, attribute-value>
- **Lexeme**
 - the sequence of input characters that a token represents.
- token-name: type of tokens

Pattern

- A **pattern** is a description of the form that the lexemes of a token may take.
- type of tokens

keywords	if, for, while ...	
identifiers	x, n1, getToken ...	标识符对应的字符串具有什么样的结构特点?
constants	10, 0.618, 1.23E-10 ...	整型/浮点型常量对应的字符串具有什么样的结构特点?
operators	+, -, *, / ...	
delimiters	; ...	

- In a lexical analyzer, token types are typically represented by integer codes.

```
typedef enum
{
    IF, THEN, ELSE, END, REPEAT, UNTIL, READ, WRITE,
    ID, NUM,
    ASSIGN, EQ, LT, PLUS, MINUS, TIMES, OVER, LPAREN, RPAREN, SEMI
} TokenType;
```

3.1.3 Attributes for Tokens

- `<token-name, attribute-value>`

```
if (n>0) position = initial + rate * 60;
```

Token	Token
<code>< if, _ ></code>	<code>< IF, _ ></code>
<code>< (, _ ></code>	<code>< LPAR, _ ></code>
<code>< id, "n" ></code>	<code>< ID, pointer to the symbol-table entry for n ></code>
<code>< >, _ ></code>	<code>< GT, _ ></code>
<code>< number, 0 ></code>	<code>< NUM, 0 ></code>
<code><), _ ></code>	<code>< RPAR, _ ></code>
...	...

3.1.4 Lexical Errors

`if (a>=0) a--;` 这是一条for语句

`if (2a>=0) a--;`

`fi (a>=0) a--;`

The simplest recovery strategy is "[panic mode](#)" recovery.

Exercise 3.1.1

```
float limitedSquare(float x)
{
    /* returns x-squared, but never more than 100 */
    return (x<=-10.0 || x>=10.0) ? 100 : x*x ;
}
```

3.2 Input Buffering

Problem & Solution

- 不管缓冲区设置得多大都不能保证token完整出现在缓冲区中。
- 使用两个交替读入的缓冲区（缓冲区对）

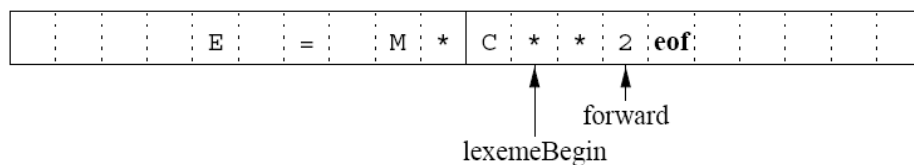


Figure 3.3: Using a pair of input buffers

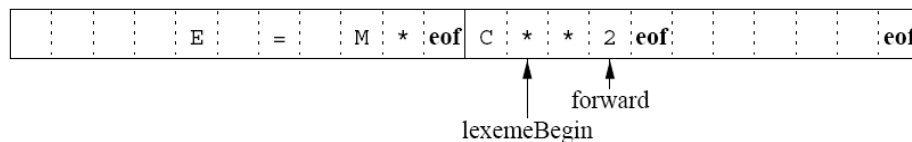


Figure 3.4: Sentinels at the end of each buffer

```
switch ( *forward++ ) {  
    case eof:  
        if (forward is at end of first buffer ) {  
            reload second buffer;  
            forward = beginning of second buffer;  
        }  
        else if (forward is at end of second buffer ) {  
            reload first buffer;  
            forward = beginning of first buffer;  
        }  
        else /* eof within a buffer marks the end of input */  
            terminate lexical analysis;  
        break;  
    Cases for the other characters  
}
```

Figure 3.5: Lookahead code with sentinels

Implementation of a Tiny Scanner

- 逐行读入
- 限制每一行的长度

```
/* BUFLen = length of the input buffer for source code lines */
```

```
#define BUFLen 256
```

```
static char lineBuf[BUFLen]; /* holds the current line */
```

- 限制TOKEN的长度

```
/* MAXTOKENLEN is the maximum size of a token */
```

```
#define MAXTOKENLEN 40
```

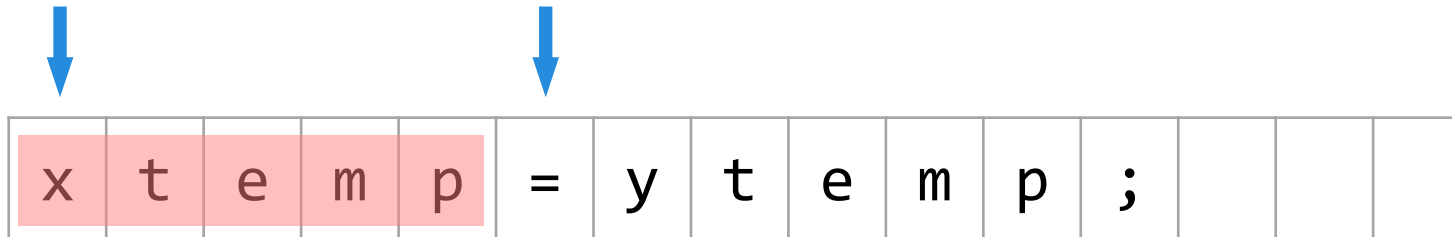
```
/* lexeme of identifier or reserved word */
```

```
char tokenString[MAXTOKENLEN+1];
```

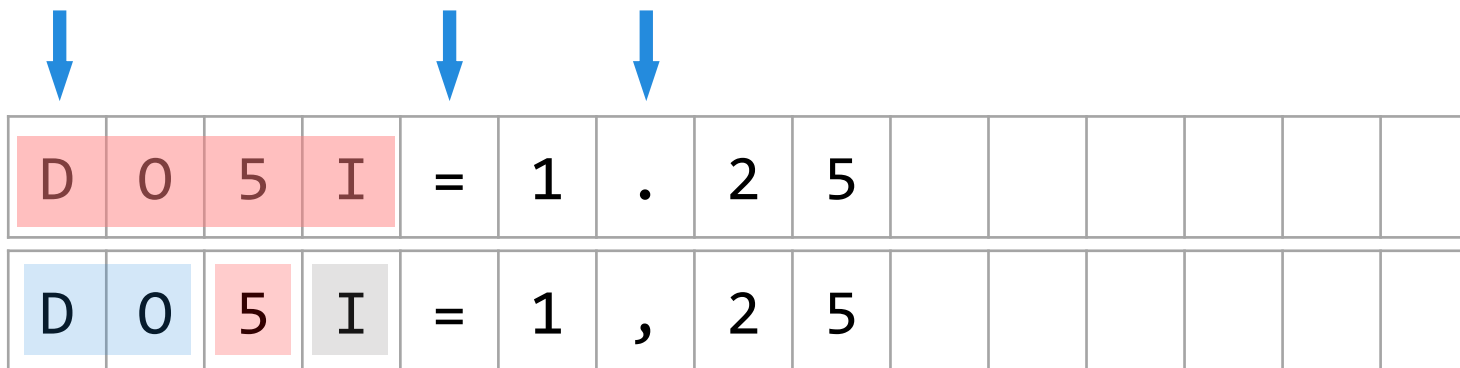


Lookahead

- single-character lookahead



- more than single-character lookahead
 - buffering of input characters and marking places for backtracking



3.3 Specification of Tokens

- 正则表达式是一种描述词素模式的形式化表示方法。

3.3.1 Strings and Languages

- An **alphabet** Σ is any finite set of symbols.
 - 汉语拼音字母表 • $\Sigma = \{ 0, 1 \}$ • $\Sigma = \text{ASCII}$
 - 26个英文字母表 • $\Sigma = \{ a, b, c \}$ • $\Sigma = \text{Unicode}$
- A **string** over an alphabet is a finite sequence of symbols drawn from that alphabet.
- A **language** is any countable set of strings over some fixed alphabet.

Σ	String	Language
$\{ 0, 1 \}$	0, 1, 00, 01, 10, 11, 000, 111	$\{ 00, 01, 10, 11 \}$ or $\{ \varepsilon, 0, 1, 00, 01, 10, 11, 000, \dots \}$
$\{ a, b \}$	ab, aabb, aaabbb, aaaabbbb, ...	$\{ ab, aabb, aaabbb, aaaabbbb, aaaaabbbbb \dots \}$

- The empty string ε , is the string of length zero.
- Difference between $\{ \varepsilon \}$ and $\{ \}$; $\Phi = \{ \}$

3.3.2 Operations on Languages

- Operations on strings
 - Concatenation: If $x = \text{dog}$ and $y = \text{house}$, then $xy = \text{doghouse}$. $\varepsilon s = s\varepsilon = s$
 - Exponentiation: Define $s^0 = \varepsilon$, $s^i = s^{i-1}s$ ($i > 0$). $s^1 = s$, $s^2 = ss$, $s^3 = sss$
- Operations on languages
 - Union
 - Concatenation
 - Closure
 - For example, $L = \{ab, c\}$, $M = \{a, bc\}$
 - $L \cup M$
 - LM
 - $L^* = L^0 \cup L^1 \cup L^2 \cup L^3 \cup \dots$
 - $L^+ = LL^* = L^1 \cup L^2 \cup L^3 \cup \dots$

OPERATION	DEFINITION AND NOTATION
<i>Union of L and M</i>	$L \cup M = \{s \mid s \text{ is in } L \text{ or } s \text{ is in } M\}$
<i>Concatenation of L and M</i>	$LM = \{st \mid s \text{ is in } L \text{ and } t \text{ is in } M\}$
<i>Kleene closure of L</i>	$L^* = \bigcup_{i=0}^{\infty} L^i$
<i>Positive closure of L</i>	$L^+ = \bigcup_{i=1}^{\infty} L^i$

Figure 3.6: Definitions of operations on languages

Example 3.3

- $L = \{ A, B, \dots, Z, a, b, \dots, z \}$, $D = \{ 0, 1, \dots, 9 \}$
 - L and D are alphabets. **or** L and D are languages.
 - New languages:
 1. LUD • $\{ A, \dots, Z, a, \dots, z, 0, \dots, 9 \}$ **or** • 62个长度为1的字母串和数字串的集合
 2. LD • 520个长度为2的字母串和数字串的集合, 且每个串都是字母在前, 数字在后
 3. L^4 • 长度为4的字母串的集合
 4. L^* • 所有由字母构成的串的集合, 包括空串
 5. D^+ • 一个或多个数字构成的串的集合
 6. $L(LUD)^*$ • It is almost exactly to describe the **set of valid C identifiers**.



$L = \{ A, B, \dots, Z, a, b, \dots, z, _ \}$

$L(L \mid D)^*$

3.3.3 Regular Expressions

- Regular expressions vs. Arithmetic expressions

Basic REs	$a \in \Sigma, \quad L(\mathbf{a}) = \{ a \}$
	$\varepsilon, \quad L(\varepsilon) = \{ \varepsilon \}$
	$\Phi, \quad L(\Phi) = \{ \}$
Operations	r and s are regular expressions, $L(r)$ and $L(s)$
	choice, $r \mid s, \quad L(r \mid s) = L(r) \cup L(s)$
	concatenation, $rs, \quad L(rs) = L(r) L(s)$
	closure, $r^*, \quad L(r^*) = L(r)^*$
Parentheses	$(), \quad (r), \quad L((r)) = L(r)$

Rules for the Definition

Basis	1. ε is a regular expression, and $L(\varepsilon) = \{ \varepsilon \}$.
	2. If a is a symbol in Σ , then $\mathbf{a}$ is a RE, and $L(\mathbf{a}) = \{ a \}$.
Induction	r and s are regular expressions, $L(r)$ and $L(s)$
	1. $(r) \mid (s)$, $L(r) \cup L(s)$
	2. $(r)(s)$, $L(r)L(s)$
	3. $(r)^*$, $L(r)^*$
	4. (r) , $L(r)$
Conventions	Precedence: $*$ $>$ concatenation $>$ $\mid$
	Association: left associative
	For example, $(a)\mid((b)^*(c))$ can be replaced by $a\mid b^*c$.

Example 3.4

- $\Sigma = \{ a, b \}$

regular expression	language	} regular set regular language
a	{ a }	
a b or b a	{ a, b }	
(a b)(a b) or (b a)(b a)	{ aa, ab, ba, bb }	
a*	{ ϵ , a, aa, ... } 任意个 a 组成的串的集合	
(a b)*	任意个 a 或 b 组成的串	
a(a b)*	Σ 上所有以 a 开头的串	
a a*b	串a和以b结尾的零个或多个a组成的串的集合	

- A language that can be defined by a regular expression is called a **regular set**.

Equivalent

- If two regular expressions r and s denote the same regular set, we say they are **equivalent** and write $r = s$.

- $a|b = b|a$
- $(a|b)(a|b) = (b|a)(b|a)$
- $b(ab)^* \neq (ba)^*b$
- $(a|b)^* \neq a^*|b^*$
- $(a|b)^* \neq (a^*|b^*)^*$
- $(a|b)^* \neq (a^*b^*)^*$

- Show that $b(ab)^* = (ba)^*b$.

$$L(b(ab)^*)$$

$$= L(b)L((ab)^*)$$

$$= L(b) (L(ab))^*$$

$$= L(b) (L(a)L(b))^*$$

$$= \{b\} \{ab\}^*$$

$$= \{b\} \{\epsilon, ab, abab, ababab, \dots\}$$

$$= \{b, bab, babab, bababab, \dots\}$$

$$\therefore L(b(ab)^*) = L((ba)^*b)$$

$$L((ba)^*b)$$

$$= L((ba)^*) L(b)$$

$$= (L(ba))^* L(b)$$

$$= (L(b)L(a))^* L(b)$$

$$= \{ba\}^* \{b\}$$

$$= \{\epsilon, ba, baba, bababa, \dots\} \{b\}$$

$$= \{b, bab, babab, bababab, \dots\}$$

$$\therefore b(ab)^* = (ba)^*b$$



Example

- $\Sigma = \{ a, b, c \}$
 1. Consider the set of all strings over this alphabet that contain exactly one b.
 2. Consider the set of all strings that contain at most one b.
 3. Consider the set of strings consists of a single b surrounded by the same number of a's.
$$S = \{b, aba, aabaa, \dots\} = \{ a^n b a^n \mid n \geq 0 \}$$
nonregular set
 4. $((b|c)^* a (b|c)^* a)^* (b|c)^*$
- All strings of English letters (including/excluding empty string)



3.3.4 Regular Definitions

- 给正则表达式命名，然后在其他正则表达式中可以直接使用这些名字，以简化表达形式。
- **Example 3.5** A regular definition for the language of C identifiers

$$letter_ \rightarrow a | b | \dots | z | A | B | \dots | Z | _$$
$$digit \rightarrow 0 | 1 | \dots | 9$$
$$id \rightarrow letter_ (letter_ | digit)^*$$
$$letter_ = a | b | \dots | z | A | B | \dots | Z | _$$
$$digit = 0 | 1 | \dots | 9$$
$$id = letter_ (letter_ | digit)^*$$

3.3.5 Extensions of Regular Expressions

- Kleene, 1950s
 - union, concatenation, Kleene closure
- Extensions

One or more instance	r^+
Zero or one instance	$r?$ $(a c)^*(b \epsilon) (a c)^* = (a c)^* b? (a c)^*$
Character classes	$0 1 2 \dots 9$ $[0-9]$
	$a b c \dots z$ $[a-z]$
	$a b c$ $[abc]$
	$a b c \dots z A B \dots Z$ $[a-zA-Z]$
	$[A-z]$ will not match $[A-Za-z]$.

Exercises

- Give regular expressions for the following languages:
 - All strings of decimal integers that are divisible by 5 (can start with 0).
 - All strings of lowercase letters that begin and end in a .
 - All strings of lowercase letters in which the letters are in ascending lexicographic order.
- Describe the languages denoted by the following regular expressions:
 - $0^*10^*10^*10^*$
 - $(0|1)^*0(0|1)(0|1)$
 - $((\epsilon|a)b^*)^*$

REs for Programming Language Tokens

- Type of tokens
 - identifiers, keywords, operators, delimiters and constants
 - whitespace: newline, blank, tab, comment
- Identifiers $id = letter(letter \mid digit)^*$
- Keywords $reserved = \mathbf{if} \mid \mathbf{then} \mid \mathbf{else} \mid \mathbf{end} \mid \mathbf{repeat} \mid \mathbf{until} \mid \mathbf{read} \mid \mathbf{write}$
- Unsigned numbers
 - strings such as 5280, 0.01234, 6.336E4, or 1.89E-4.
- **Example 3.6 vs. Example 3.7**

REs for Programming Language Tokens

- Comments

{ this is a Tiny comment }

// this is a C comment

/* this is a C comment */ $ba(\sim(ab))^*ab$

- $\sim$ restricted to single character

- one solution for $\sim(ab)$: $b^*(a^*\sim(a|b)b^*)^*a^*$

- Difficult to read and to prove correct.

- So complicated that it is almost never written in practice.

- This case is usually handled by *ad hoc* methods in actual scanners.

- Whitespace

$whitespace = (newline | blank | tab | comment)^+$



Example

- $\Sigma = \{ a, b, c \}$.

All strings over this alphabet that contain no two consecutive b's.

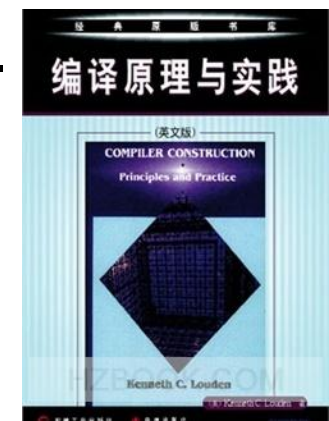
- All strings of English letters that contain no two consecutive b's.

- $\Sigma = \{ a, b \}$

All strings over this alphabet that contain no three consecutive b's.

- $\Sigma = \{ 0, 1 \}$

All strings over this alphabet that each 1 followed by one 0 or more 0s.



Exercise 3.3.4: Most languages are *case sensitive*, so keywords can be written only one way, and the regular expressions describing their lexeme is very simple. However, some languages, like SQL, are *case insensitive*, so a keyword can be written either in lowercase or in uppercase, or in any mixture of cases. Thus, the SQL keyword **SELECT** can also be written **select**, **Select**, or **sElEcT**, for instance. Show how to write a regular expression for a keyword in a case-insensitive language. Illustrate the idea by writing the expression for “select” in SQL.